database-logical-design

Documentación del Diseño Lógico de Base de Datos - Sistema ZWL

1. Introducción

El diseño lógico representa la estructura detallada de la base de datos, transformando el modelo conceptual en un esquema implementable en MySQL. Este documento describe las tablas, columnas, tipos de datos, restricciones e índices que conforman el esquema lógico de la base de datos ZWL.

2. Esquema de la Base de Datos

Nombre de la base de datos: `zwl`
Motor: MySQL con InnoDB
Juego de caracteres: utf8mb4
Cotejamiento: utf8mb4_unicode_ci

---

3. Especificación de Tablas

3.1 Tabla: `usuarios`

Propósito: Almacena la información de los usuarios del sistema.

Columna	Tipo	Nulo	Clave	Default	Extra
id	INT	NO	PRI	-	AUTO_INCREM
nombre	VARCHAR(100)	NO	-	-	-
email	VARCHAR(100)	NO	UNI	-	-
created_at	TIMESTAMP	YES	-	CURRENT_TIMESTAMP	-
updated_at	TIMESTAMP	YES	-	CURRENT_TIMESTAMP	ON UPDATE CURRENT_TIM

Índices:

- PRIMARY KEY (`id`)

- UNIQUE KEY (`email`)

3.2 Tabla: `productos`

Propósito: Catálogo de productos para venta e inventario.

Columna	Tipo	Nulo	Clave	Default	Extra
id	INT	NO	PRI	-	AUT
nombre	VARCHAR(150)	NO	-	-	-
categoria	VARCHAR(50)	NO	MUL	-	-
precio	DECIMAL(10,2)	NO	-	0.00	-
stock	INT	NO	-	0	-
stock_minimo	INT	NO	-	5	-
estado	ENUM('OK','Crítico','Sin stock')	YES	-	'OK'	-
created_at	TIMESTAMP	YES	-	CURRENT_TIMESTAMP	-
updated_at	TIMESTAMP	YES	-	CURRENT_TIMESTAMP	ON U CUR

Índices:

- PRIMARY KEY (`id`)
- INDEX `idx_productos_categoria` (`categoria`)

3.3 Tabla: `ventas`

Propósito: Registro de transacciones de ventas.

Columna	Tipo	Nulo	Clave	Default
id	INT	NO	PRI	-
fecha	DATETIME	NO	MUL	CURRENT_TIM
total	DECIMAL(10,2)	NO	-	0.00
usuario_id	INT	YES	MUL	-
estado	ENUM('completada','pendiente','cancelada')	YES	-	'completada'
created_at	TIMESTAMP	YES	-	CURRENT_TIM
updated_at	TIMESTAMP	YES	-	CURRENT_TIM

Índices:

- PRIMARY KEY (`id`)
- INDEX `idx_ventas_fecha` (`fecha`)
- INDEX `idx_ventas_usuario` (`usuario_id`)
- FOREIGN KEY (`usuario_id`) REFERENCES `usuarios` (`id`) ON DELETE SET NULL

3.4 Tabla: `detalle_ventas`

Propósito: Productos incluidos en cada venta.

Columna	Tipo	Nulo	Clave	Default	Extra
id	INT	NO	PRI	-	AUTO_INCR
venta_id	INT	NO	MUL	-	-
producto_id	INT	NO	MUL	-	-
cantidad	INT	NO	-	1	-
precio_unitario	DECIMAL(10,2)	NO	-	-	-

Columna	Tipo	Nulo	Clave	Default	Extra
subtotal	DECIMAL(10,2)	NO	-	-	-
created_at	TIMESTAMP	YES	-	CURRENT_TIMESTAMP	-

Índices:

- PRIMARY KEY (`id`)
- INDEX `idx_detalle_ventas_venta` (`venta_id`)
- INDEX `idx_detalle_ventas_producto` (`producto_id`)
- FOREIGN KEY (`venta_id`) REFERENCES `ventas` (`id`) ON DELETE CASCADE
- FOREIGN KEY (`producto_id`) REFERENCES `productos` (`id`) ON DELETE RESTRICT

3.5 Tabla: `proveedores`

Propósito: Información de empresas proveedoras.

Columna	Tipo	Nulo	Clave	Default	Extra
id	INT	NO	PRI	-	AUTO_INCREM
nombre	VARCHAR(150)	NO	-	-	-
contacto	VARCHAR(100)	YES	-	-	-
email	VARCHAR(100)	YES	-	-	-
telefono	VARCHAR(20)	YES	-	-	-
created_at	TIMESTAMP	YES	-	CURRENT_TIMESTAMP	-
updated_at	TIMESTAMP	YES	-	CURRENT_TIMESTAMP	ON UPDATE CURRENT_TIM

Índices:

- PRIMARY KEY (`id`)

3.6 Tabla: `solicitudes`

Propósito: Solicitudes de pedidos a proveedores.

Columna	Tipo	Nulo	Clave	Default
id	INT	NO	PRI	-
codigo	VARCHAR(20)	NO	UNI	-
proveedor_id	INT	NO	MUL	-
fecha	DATE	NO	MUL	-
estado	ENUM('Pendiente','Recibida','Cancelada')	YES	-	'Pendiente'
usuario_id	INT	YES	-	-
created_at	TIMESTAMP	YES	-	CURRENT_TIM
updated_at	TIMESTAMP	YES	-	CURRENT_TIM

Índices:

- PRIMARY KEY (`id`)
- UNIQUE KEY (`codigo`)
- INDEX `idx_solicitudes_proveedor` (`proveedor_id`)
- INDEX `idx_solicitudes_fecha` (`fecha`)
- FOREIGN KEY (`proveedor_id`) REFERENCES `proveedores` (`id`) ON DELETE RESTRICT
- FOREIGN KEY (`usuario_id`) REFERENCES `usuarios` (`id`) ON DELETE SET NULL

3.7 Tabla: `activos`

Propósito: Activos fijos de la empresa.

Columna	Tipo	Nulo	Clave	Default
id	INT	NO	PRI	-

Columna	Tipo	Nulo	Clave	Default
nombre	VARCHAR(150)	NO	-	-
tipo	ENUM('Equipos','Herramientas','Licencias')	NO	-	-
estado	ENUM('Activo','Mantenimiento','Vencida')	YES	-	'Activo'
fecha_adquisicion	DATE	YES	-	-
fecha_vencimiento	DATE	YES	-	-
created_at	TIMESTAMP	YES	-	CURREN
updated_at	TIMESTAMP	YES	-	CURREN

Índices:

- PRIMARY KEY (`id`)

3.8 Tabla: `estaciones_cyber`

Propósito: Estaciones de trabajo del cybercafé.

Columna	Tipo	Nulo	Clave	Default
id	INT	NO	PRI	-
nombre	VARCHAR(50)	NO	UNI	-
estado	ENUM('Disponible','Ocupada','Mantenimiento')	YES	-	'Disponible'
tipo	VARCHAR(50)	YES	-	-
created_at	TIMESTAMP	YES	-	CURRENT_T
updated_at	TIMESTAMP	YES	-	CURRENT_T

Índices:

- PRIMARY KEY (`id`)

- UNIQUE KEY (`nombre`)

3.9 Tabla: `sesiones_cyber`

Propósito: Registro de sesiones en el cybercafé.

Columna	Tipo	Nulo	Clave	Default	Extra
id	INT	NO	PRI	-	AUTO_IN
estacion_id	INT	NO	MUL	-	-
usuario_id	INT	YES	-	-	-
hora_inicio	DATETIME	NO	-	CURRENT_TIMESTAMP	-
hora_fin	DATETIME	YES	-	-	-
duracion_minutos	INT	YES	-	-	-
costo	DECIMAL(10,2)	YES	-	-	-
created_at	TIMESTAMP	YES	-	CURRENT_TIMESTAMP	-
updated_at	TIMESTAMP	YES	-	CURRENT_TIMESTAMP	ON UPDA CURREN

Índices:

- PRIMARY KEY (`id`)
- INDEX `idx_sesiones_estacion` (`estacion_id`)
- FOREIGN KEY (`estacion_id`) REFERENCES `estaciones_cyber` (`id`) ON DELETE RESTRICT
- FOREIGN KEY (`usuario_id`) REFERENCES `usuarios` (`id`) ON DELETE SET NULL

3.10 Tabla: `movimientos_stock`

Propósito: Historial de movimientos de inventario.

Columna	Tipo	Nulo	Clave	Default
id	INT	NO	PRI	-
producto_id	INT	NO	MUL	-
tipo	ENUM('entrada','salida','ajuste')	NO	-	-
cantidad	INT	NO	-	-
stock_anterior	INT	NO	-	-
stock_nuevo	INT	NO	-	-
fecha	DATETIME	NO	MUL	CURRENT_TIMESTAMP
usuario_id	INT	YES	-	-
motivo	VARCHAR(255)	YES	-	-
created_at	TIMESTAMP	YES	-	CURRENT_TIMESTAMP

Índices:

- PRIMARY KEY (`id`)
- INDEX `idx_movimientos_producto` (`producto_id`)
- INDEX `idx_movimientos_fecha` (`fecha`)
- FOREIGN KEY (`producto_id`) REFERENCES `productos` (`id`) ON DELETE RESTRICT
- FOREIGN KEY (`usuario_id`) REFERENCES `usuarios` (`id`) ON DELETE SET NULL

---

4. Diagrama de Esquema Lógico (SQL)

```
-- Esquema visual de relaciones

usuarios (id) ──────────────────────────────────┐
    │                                           │
    │                                           │
    ├──> ventas (id, usuario_id)                │
    │       │                                   │
    │       └──> detalle_ventas (id, venta_id, producto_id)
```

```
    │               │                          │
    │               └──────────────────────────┤
    │                                          │
    ├──> solicitudes (id, usuario_id)          │
    │       │                                  │
    │       └──> proveedores (id) <────────────┘
    │
    ├──> sesiones_cyber (id, usuario_id)
    │       │
    │       └──> estaciones_cyber (id)
    │
    └──> movimientos_stock (id, usuario_id)
            │
            └──> productos (id)
```

---

5. Integridad Referencial (Foreign Keys)

Tabla Hijo	Columna FK	Tabla Padre	Columna Padre	ON DELETE	ON UPDATE
ventas	usuario_id	usuarios	id	SET NULL	CASCA
detalle_ventas	venta_id	ventas	id	CASCADE	CASCA
detalle_ventas	producto_id	productos	id	RESTRICT	CASCA
solicitudes	proveedor_id	proveedores	id	RESTRICT	CASCA
solicitudes	usuario_id	usuarios	id	SET NULL	CASCA
sesiones_cyber	estacion_id	estaciones_cyber	id	RESTRICT	CASCA
sesiones_cyber	usuario_id	usuarios	id	SET NULL	CASCA
movimientos_stock	producto_id	productos	id	RESTRICT	CASCA
movimientos_stock	usuario_id	usuarios	id	SET NULL	CASCA

---

6. Tipos de Datos Utilizados

6.1 Tipos Numéricos

- **INT**: Identificadores, cantidades, duraciones
- **DECIMAL(10,2)**: Valores monetarios (precios, totales, costos)

6.2 Tipos de Cadena

- **VARCHAR(n)**: Campos de longitud variable (nombres, emails, teléfonos)

- **VARCHAR(20-50)**: Códigos cortos y nombres de estaciones
- **VARCHAR(100-150)**: Nombres y descripciones
- **VARCHAR(255)**: Motivos y textos largos

6.3 Tipos ENUM

- Estados de entidades
- Tipos de movimientos
- Categorías cerradas

6.4 Tipos Fecha/Hora

- **DATE**: Fechas sin hora (fecha_adquisicion, fecha_vencimiento)
- **DATETIME**: Fechas con hora (fecha venta, hora_inicio, hora_fin)
- **TIMESTAMP**: Control de auditoría (created_at, updated_at)

---

7. Índices y Rendimiento

7.1 Índices Primarios

Todas las tablas tienen un `PRIMARY KEY` sobre la columna `id` (AUTO_INCREMENT).

7.2 Índices Únicos

- `usuarios.email` : Evita duplicidad de correos
- `solicitudes.codigo` : Garantiza códigos únicos
- `estaciones_cyber.nombre` : Evita nombres duplicados de estaciones

7.3 Índices Secundarios (Búsqueda)

Índice	Tabla	Columna(s)	Propósito
idx_productos_categoria	productos	categoria	Filtrar por categoría
idx_ventas_fecha	ventas	fecha	Reportes por fecha
idx_ventas_usuario	ventas	usuario_id	Ventas por usuario
idx_detalle_ventas_venta	detalle_ventas	venta_id	Detalles de una venta
idx_detalle_ventas_producto	detalle_ventas	producto_id	Historial de producto
idx_solicitudes_proveedor	solicitudes	proveedor_id	Solicitudes por proveedor
idx_solicitudes_fecha	solicitudes	fecha	Reportes por fecha
idx_sesiones_estacion	sesiones_cyber	estacion_id	Sesiones por estación

Índice	Tabla	Columna(s)	Propósito
idx_movimientos_producto	movimientos_stock	producto_id	Movimientos por producto
idx_movimientos_fecha	movimientos_stock	fecha	Historial por fecha

---

8. Restricciones de Integridad

8.1 Restricciones de Clave Primaria (PK)

- Todas las tablas tienen una PK simple sobre `id`
- Garantiza identificación única de registros

8.2 Restricciones de Clave Foránea (FK)

- Mantienen la integridad referencial entre tablas relacionadas
- Evitan registros huérfanos

8.3 Restricciones UNIQUE

- `usuarios.email`: Un usuario por correo
- `solicitudes.codigo`: Códigos de solicitud únicos
- `estaciones_cyber.nombre`: Nombres de estación únicos

8.4 Restricciones NOT NULL

- Identificadores (id): Siempre requeridos
- Campos obligatorios: nombre, email, cantidad, precio, etc.
- Campos opcionales permiten NULL: contacto, email proveedor, usuario_id en sesiones

8.5 Restricciones CHECK (vía ENUM)

- Estados válidos para cada entidad
- Tipos de movimientos permitidos
- Categorías cerradas

---

9. Políticas de Eliminación (ON DELETE)

9.1 SET NULL

Uso: Cuando se elimina el registro padre, se conserva el hijo pero se anula la referencia.

- `ventas.usuario_id` : Si se elimina un usuario, sus ventas se mantienen pero sin referencia
- `solicitudes.usuario_id` : Las solicitudes se mantienen sin referencia al usuario
- `sesiones_cyber.usuario_id` : Las sesiones se mantienen sin referencia al usuario
- `movimientos_stock.usuario_id` : El historial se mantiene sin referencia al usuario

9.2 CASCADE

Uso: Al eliminar el padre, se eliminan automáticamente los hijos.

- `detalle_ventas.venta_id` : Al eliminar una venta, se eliminan sus detalles

9.3 RESTRICT

Uso: Impide eliminar el padre si tiene hijos relacionados.

- `detalle_ventas.producto_id` : No se puede eliminar un producto con ventas
- `movimientos_stock.producto_id` : No se puede eliminar un producto con movimientos
- `solicitudes.proveedor_id` : No se puede eliminar un proveedor con solicitudes
- `sesiones_cyber.estacion_id` : No se puede eliminar una estación con sesiones

---

10. Consultas SQL Comunes

10.1 Obtener todas las ventas con su usuario

```sql
SELECT v.id, v.fecha, v.total, v.estado, u.nombre as usuario
FROM ventas v
LEFT JOIN usuarios u ON v.usuario_id = u.id
ORDER BY v.fecha DESC;
```

10.2 Detalle de una venta con información del producto

```sql
SELECT dv.cantidad, p.nombre, dv.precio_unitario, dv.subtotal
FROM detalle_ventas dv
INNER JOIN productos p ON dv.producto_id = p.id
WHERE dv.venta_id = ?;
```

10.3 Productos con stock crítico o agotado

```sql
SELECT id, nombre, stock, stock_minimo, estado
FROM productos
WHERE stock <= stock_minimo
ORDER BY stock ASC;
```

10.4 Historial de movimientos de un producto

```sql
SELECT ms.fecha, ms.tipo, ms.cantidad, ms.stock_anterior, ms.stock_nuevo,
u.nombre as usuario
FROM movimientos_stock ms
LEFT JOIN usuarios u ON ms.usuario_id = u.id
WHERE ms.producto_id = ?
ORDER BY ms.fecha DESC;
```

10.5 Sesiones activas en el cybercafé

```sql
SELECT s.id, e.nombre as estacion, s.hora_inicio, u.nombre as usuario
FROM sesiones_cyber s
INNER JOIN estaciones_cyber e ON s.estacion_id = e.id
LEFT JOIN usuarios u ON s.usuario_id = u.id
WHERE s.hora_fin IS NULL;
```

10.6 Reporte de ventas por período

```sql
SELECT
    DATE(fecha) as dia,
    COUNT(*) as total_ventas,
    SUM(total) as monto_total
FROM ventas
WHERE fecha BETWEEN ? AND ?
GROUP BY DATE(fecha)
ORDER BY dia DESC;
```

---

11. Normalización

11.1 Primera Forma Normal (1NF)

- Todos los valores son atómicos
- No hay grupos repetidos
- Cada fila tiene una clave primaria única

11.2 Segunda Forma Normal (2NF)

- Está en 1NF
- Todas las columnas no clave dependen completamente de la clave primaria

11.3 Tercera Forma Normal (3NF)

- Está en 2NF
- No hay dependencias transitivas
- Cada tabla representa una entidad única

---

12. Consideraciones de Diseño

12.1 Uso de TIMESTAMP vs DATETIME

- `created_at` y `updated_at`: TIMESTAMP para auditoría automática
- `fecha` en ventas/solicitudes: DATETIME para fechas independientes de zona horaria
- `hora_inicio/fin` en sesiones: DATETIME para precisión de tiempo

12.2 Valores por Defecto

- Estados: Siempre tienen un valor inicial válido
- Fechas: CURRENT_TIMESTAMP donde aplique
- Stock: 0 por defecto para nuevos productos
- Precios: 0.00 para evitar NULL en cálculos

12.3 Campos Opcionales (NULL)

- `usuario_id` en ventas, sesiones, movimientos: Permite operaciones anónimas
- Datos de contacto en proveedores: No todos los proveedores tienen todos los datos
- `hora_fin` en sesiones: NULL indica sesión activa

---

13. Escalabilidad y Mantenimiento

13.1 Particionamiento Futuro

Para tablas que crecen rápido (`movimientos_stock` , `sesiones_cyber`), considerar:

- Particionamiento por rango de fechas
- Archivado de datos antiguos

13.2 Índices Adicionales

Según necesidades de consulta, se pueden agregar:

- Índices compuestos para consultas frecuentes
- Índices en campos de búsqueda de texto

13.3 Auditoría

- `created_at` y `updated_at` en todas las tablas principales
- `movimientos_stock` actúa como auditoría de inventario
- Las FK con SET NULL preservan el historial

---

14. Conclusión

El diseño lógico de la base de datos ZWL implementa un esquema robusto, normalizado y optimizado para el rendimiento. El uso adecuado de restricciones, índices y políticas de integridad referencial garantiza la consistencia de los datos y facilita el mantenimiento a largo plazo.